

WORKSHOP

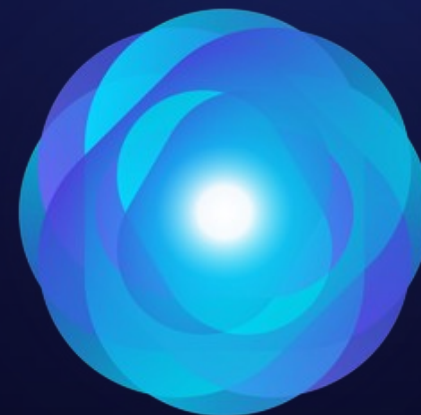
AgentOps: Full Production Lifecycle

Build, deploy, scale and monitor a multi-agent system

SPEAKER

Kartik Nighania

MLOps Engineer · Typewise



Scan This QR Code



Everything for today's session in one place.



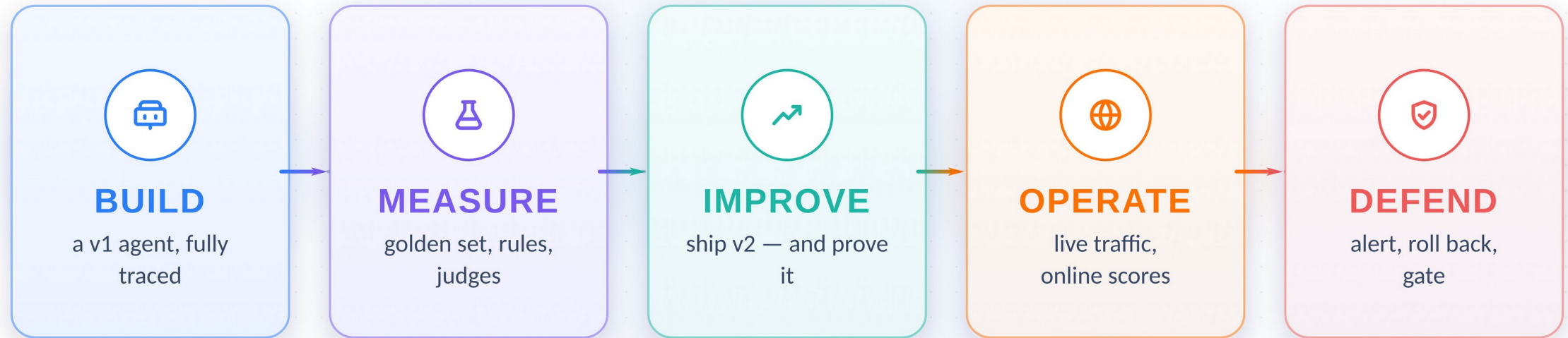
github.com/kartik-nighania/data-hack-summit-2026

- **Scan to access session resources**
one link, open on your phone or your laptop
- **The Colab notebook**
the exact notebook we run today, top to bottom
- **Your Langfuse keys**
cloud.langfuse.com → Settings ► API Keys

The Arc of the Day



THE ARC OF THE DAY



The notebook runs top to bottom in exactly this order.



Nothing here is a toy step

a bad prompt shipped to production gets caught, rolled back and blocked from merging — live, in this session

The Stack You'll Use Today



YOUR AGENT



LangGraph · LangChain · OpenAI
the system under observation

INSTRUMENT



Langfuse SDK v4 · OpenTelemetry
@observe · CallbackHandler · masking hooks

EVALUATE



rules · LLM judges · DeepEval
computed where your code runs, stored where your team can see it

PLATFORM



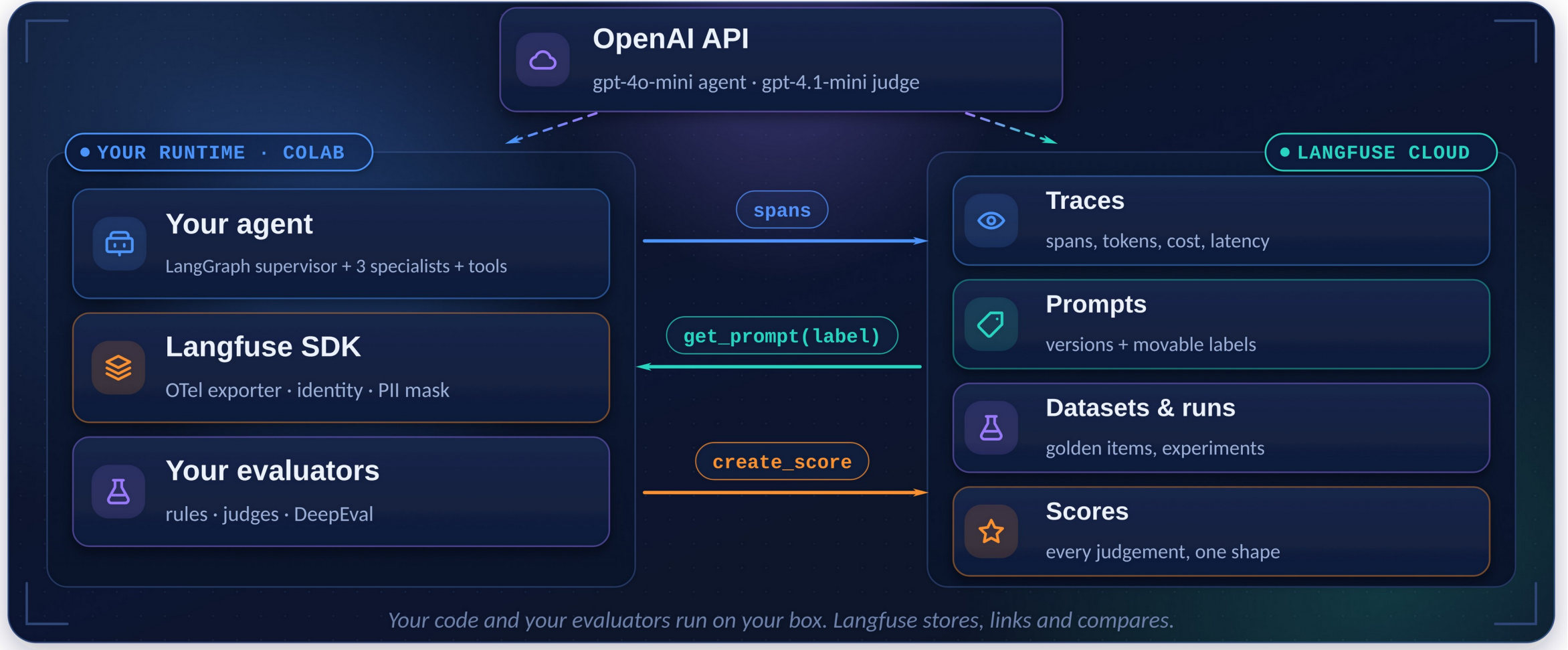
Langfuse Cloud
traces · prompts · datasets · scores · dashboards · monitors

DELIVER



GitHub Actions · experiment-action
the same evaluation, re-run as a merge gate

Where Everything Actually Runs



One Loop, Ten Modules

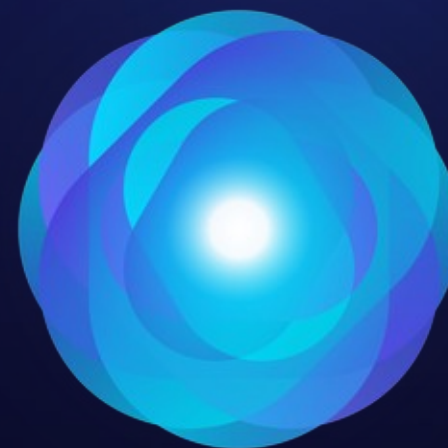


01

MODULE 0

Setup & connection check

Keys, secrets, pinned versions — and the first trace that proves the pipe is open.



Before You Run Anything



Five minutes now saves forty minutes of debugging later.

- **Langfuse keys from your own project**
cloud.langfuse.com → your project → Settings ► API Keys. Note the region host.
- **Store them as Colab secrets**
the key icon in the left sidebar — add each one and enable notebook access.
- **Versions are pinned on purpose**
langfuse 4.14.1 · langchain 1.3.14 · langgraph 1.2.9 · deepeval 4.1.1
- **Everything after this is idempotent**
re-run any cell: prompts upsert, dataset items upsert, scores overwrite by id.

4

keys, and only four

~25 s

ingestion delay before new data is queryable

100 / day

Metrics API calls on the free tier

How Your Credentials Reach the SDK



Same three keys everywhere

Colab Secrets → environment variables → an interactive prompt. Colab, a laptop or a CI runner: identical code path.

PINNED

langfuse 4.14.1

langchain 1.3.14

langgraph 1.2.9

deepeval 4.1.1

openai 2.46.0

Checkpoint · Your First Trace



If this two-node tree appears, everything else today will too.

```
module_0.py

@observe(name="connection-check")
def connection_check():
    client = OpenAI()
    resp = client.chat.completions.create(
        model=AGENT_MODEL,
        messages=[{"role": "user", "content": "Reply: pong"}],
    )
    return resp.choices[0].message.content

print("OpenAI says:", connection_check())
lf.flush()  # force the export
```

WHAT TO CALL OUT

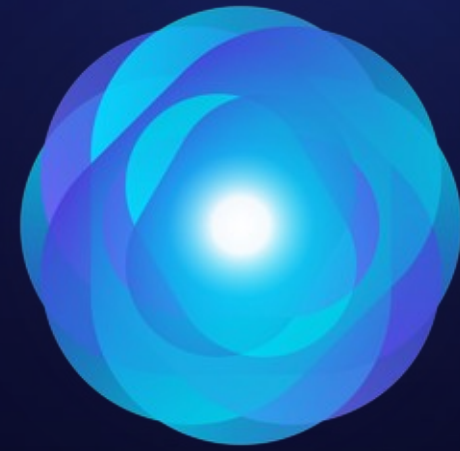
- **A decorator is the whole API**
@observe turns any function into the root span of a trace.
- **The LLM call instruments itself**
it becomes a GENERATION with model, tokens and cost.
- **flush() is not optional in a notebook**
spans are batched; the kernel can end before they ship.
- **Then wait, then refresh**
Langfuse needs ~15–30 s before new data is queryable.

02

MODULE 1

Observability foundations

Three disciplines, one data model — and the vocabulary the rest of the day depends on.



Three Disciplines, Three Questions



TRACING

What exactly happened in this request?

- Traces & observations
- Sessions & users
- Tags & environments

MONITORING

Is the system healthy right now?

- Dashboards & widgets
- Metrics API
- Monitors & alerts

EVALUATION

Is the output any good — and improving?

- Scores & score configs
- Datasets & experiments
- Judges & annotation



One data model underneath all three

the trace that lets you debug one angry customer is the row that feeds the cost dashboard, and the unit an evaluator attaches a score to

How Langfuse Organises Your Data



Everything you build today is one of these six objects — the rest is deciding which one to write, and when.

Anatomy of a Trace · and Where the Money Goes



• ONE TRACE, TOP TO BOTTOM

 meridian-support-agent	SPAN	1 842 ms
 supervisor	GENERATION	410 ms
 account_agent	AGENT	980 ms
 get_loan_summary	TOOL	12 ms
 llm call	GENERATION	930 ms
 finalize	GENERATION	190 ms

WHERE THE MONEY GOES



Step-level attribution

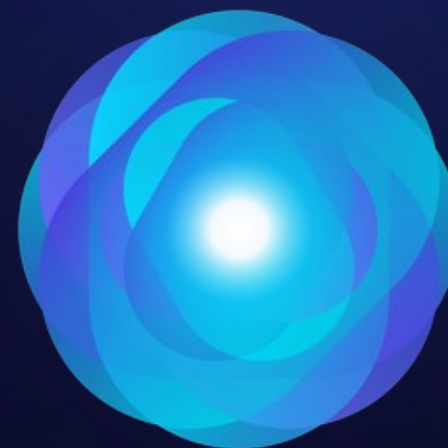
the difference between knowing and guessing

03

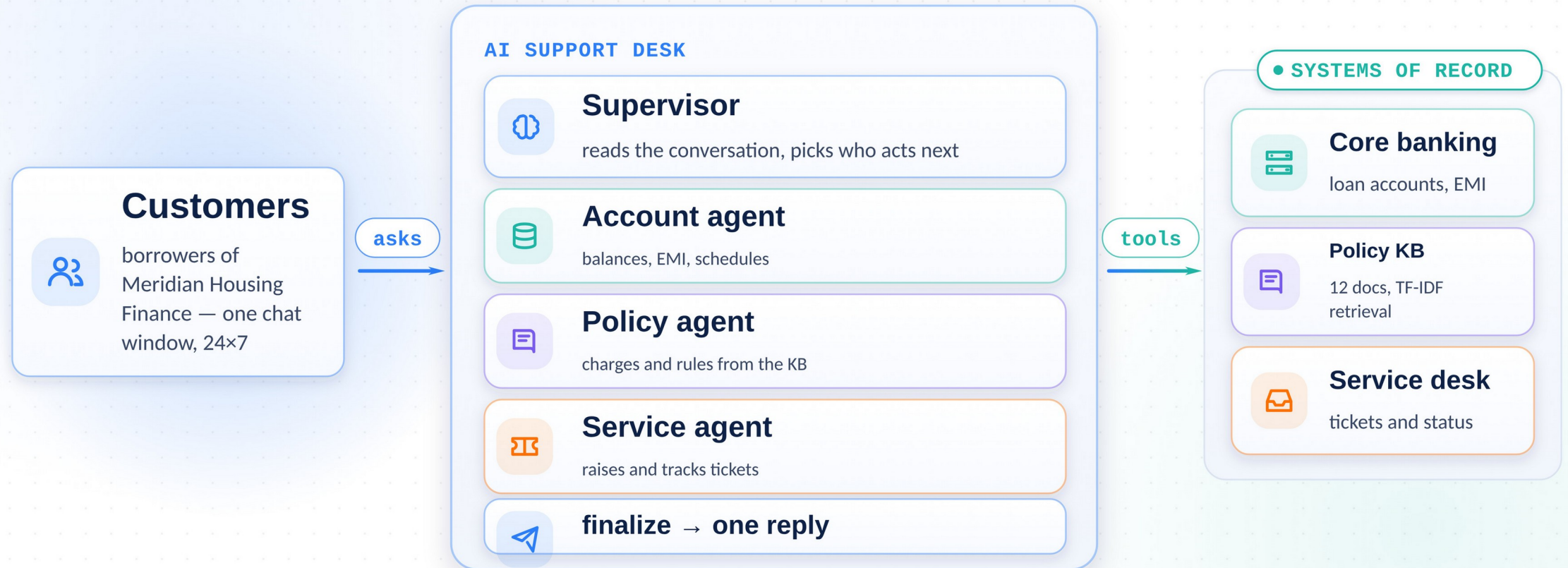
MODULE 2

Build the multi-agent, wire in Langfuse

A supervisor, three specialists, real tools — and full telemetry from the first request.

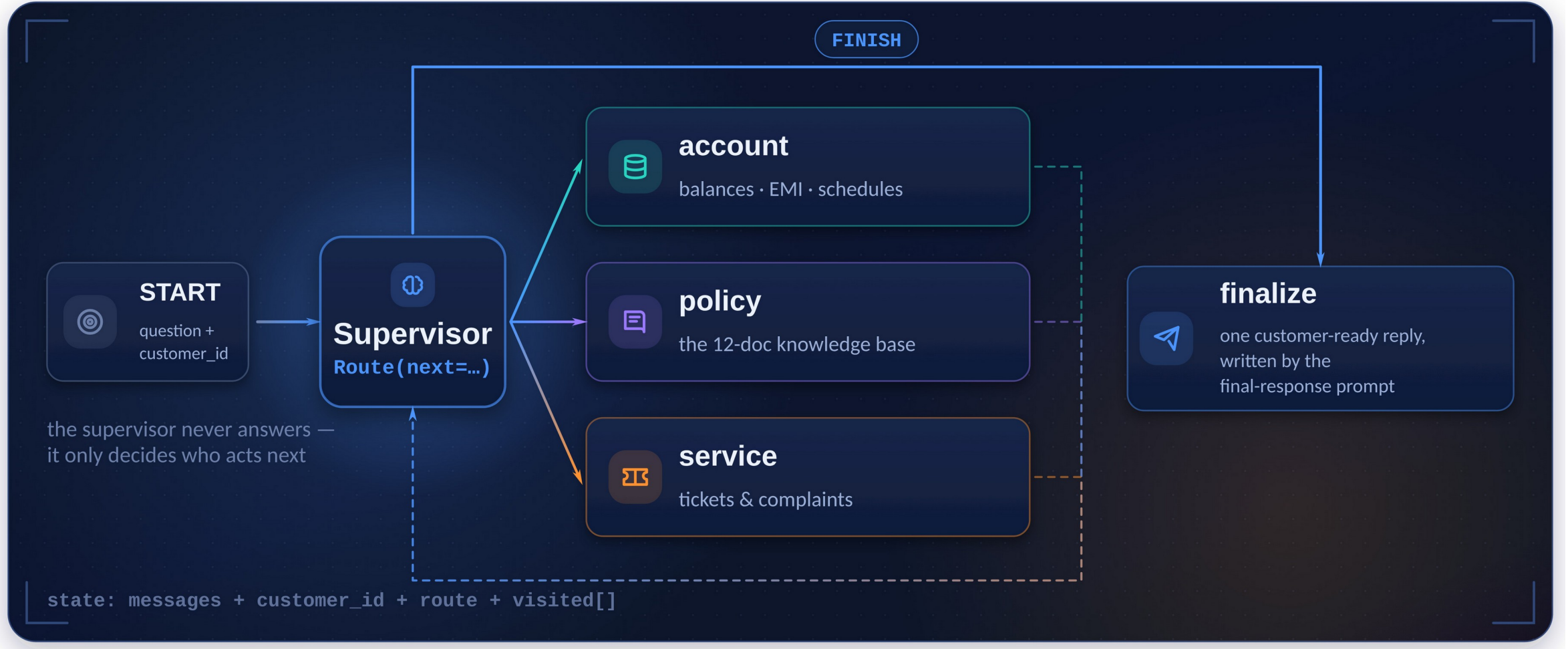


What We're Building · Meridian Housing Finance



Every customer, loan and policy here is synthetic. The architecture is the one you would actually ship.

Meridian's Agent Graph



PII · The Leak, and the One-Line Fix



WITHOUT THE HOOK

Tool result → Span attributes → Exporter → Stored trace

👁 phone: +91-9820011223 pan: ABCPS1234F

LEAKED

WITH `mask_otel_spans`

Tool result → ^{mask runs HERE} Span attributes → Exporter → Stored trace

🗿 phone: «PHONE-REDACTED» pan: «PAN-REDACTED»

REDACTED AT EXPORT

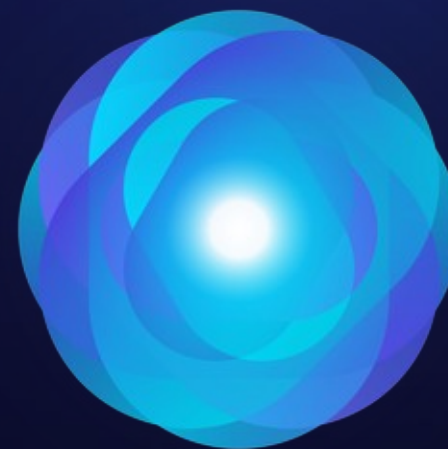
The hook patches raw OTEL attributes at export — so it also covers spans the LangChain integration created for you.

04

MODULE 3

Prompt versioning & deployment

The prompt is the behaviour of your agent. Treat it like a deployable, not a string literal.



Versions, Labels, and What 'Deploy' Means Here



IMMUTABLE VERSIONS

v1 **version 1**
flawed on purpose

v2 **version 2**
diagnosed, fixed, proven

v3 **version 3**
a 'harmless' brevity tweak

MOVABLE LABELS

 **production**
what your app fetches

 **latest**
always the newest version

 **staging**
any label you invent



Moving the label IS the deployment

the next customer request uses it — no redeploy, and rollback is the same call backwards

Resilience · Cache, Fallback, and the Link You Lose



The failure mode nobody plans for: Langfuse unreachable at cold start.

```
module_3.py

p = lf.get_prompt(
    "final-response", type="chat", cache_ttl_seconds=60,
    fallback=[
        {"role": "system", "content":
        "You are Meridian's support desk."},
        {"type": "placeholder", "name": "conversation"},
    ],
)

print(p.is_fallback)  # False while the API is reachable
```

WHAT TO CALL OUT

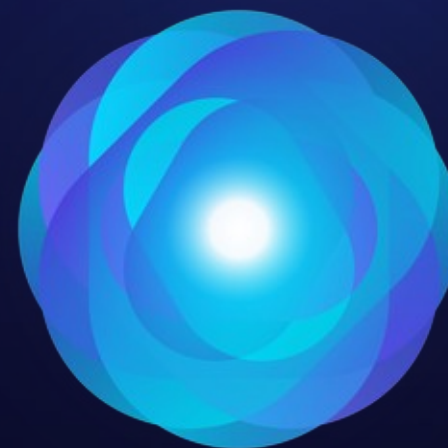
- **The cache is your latency budget**
ttl=0 in a workshop; 60+ in production, or you pay a hop per request.
- **Fallback only fires on a cold miss**
no cached copy AND the fetch failed.
- **A fallback prompt has no version**
so those generations carry no prompt ↔ trace link.
- **Alarm on it**
a silent fallback quietly deletes rows from your per-version metrics.

05

MODULE 4

Tracing & monitoring executions

Read any trace, follow a whole conversation, recognise a retry storm, slice the traffic.



One Conversation, End to End



• ONE SESSION, THREE TURNS

1

“What's the outstanding on each of my two loans?”

route: account

2

“For the top-up loan, what are the next 3 EMIs?”

route: account

3

“And if I prepay some of it, are there charges?”

route: policy

one trace per turn • all sharing the same session_id



Sessions view



three traces stitched into one thread, with combined cost and latency

Users view



user_id on every trace, so per-customer cost and activity come for free

What it answers



who costs the most • whose sessions fail • did turn two lose turn one's context

Timeouts & Retries · What Resilience Looks Like



INSIDE ONE TOOL SPAN



get_emi_schedule

3 140 ms total · 3 attempts



attempt 1 TimeoutError · 1 200 ms



ERROR



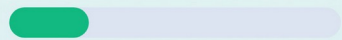
attempt 2 TimeoutError · 1 200 ms



ERROR



attempt 3 200 OK · 740 ms



OK

• HOW YOU FIND THESE



Filter level = ERROR

every trace with a failed attempt



Watch p95, not the mean

retries live in the tail



Alert on retry rate

a degrading dependency shows here first



Attribute the latency

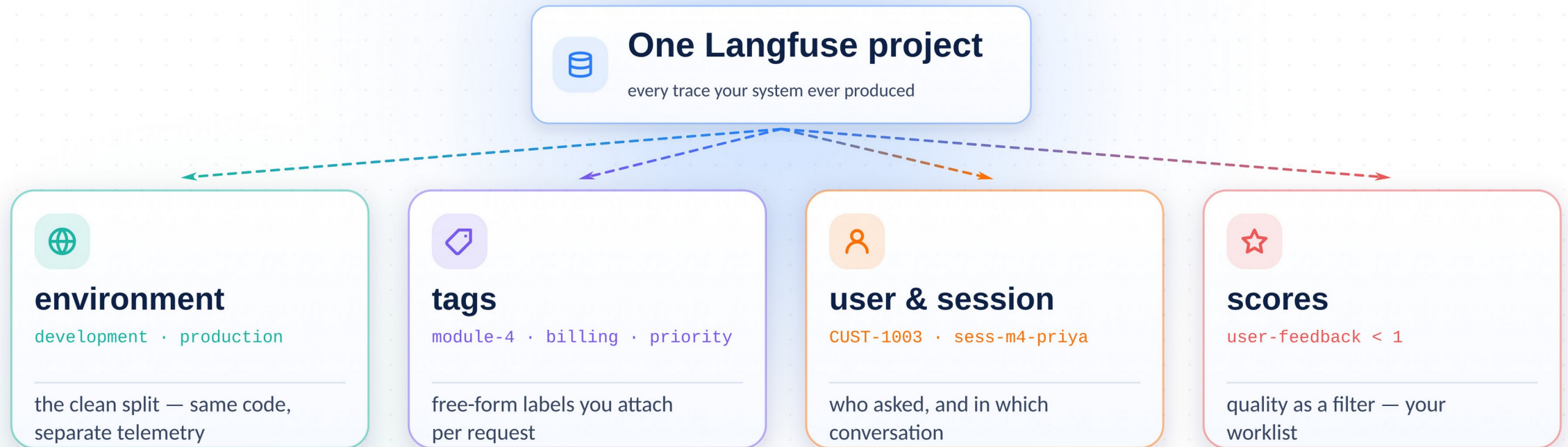
a dependency problem, not a prompt problem



Resilience is invisible until you trace it

the customer saw one slow reply — the trace shows two silent failures, a backoff and a recovery

Slicing Traffic · Tags, Environments, Filters



Environments matter more than they look

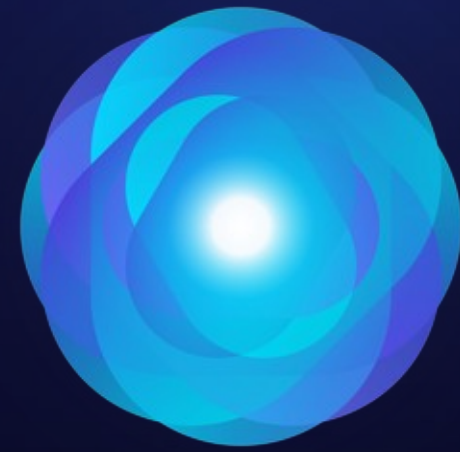
today's traffic is development; Module 7 stamps production per request. Same code, same project — every view, metric and evaluator can tell them apart.

06

MODULE 5

Capturing user feedback

Real signals from real people — attached to exactly the right trace, minutes after the fact.



The Feedback Round-Trip



no mapping table anywhere in this picture

• YOUR APPLICATION



1 · request id

req-2026-07-20-000123



2 · trace_seed

the trace is created under the derived id



3 · minutes later

the customer clicks thumbs-down

spans

create_score(trace_id)

• LANGFUSE



trace 8f3c...9ab1

found by recomputing the same id



score · user-feedback

BOOLEAN 1 / 0, with the comment



filter · feedback < 1

your unhappy-customer worklist



create_trace_id(seed=request_id) is a pure function

same seed, same trace id — in any process, on any machine, at any time

Score Configs · Pin the Schema Once



So that 'csat' means the same thing whoever writes it, forever.

module_5.py

```
CSAT_CONFIG = ensure_score_config(
    "csat", data_type="CATEGORICAL",
    categories=[
        {"label": "1-very-bad", "value": 1},
        ...
        {"label": "5-excellent", "value": 5},
    ],
)

lf.create_score(
    trace_id=create_trace_id(seed=request_id),
    name="user-feedback", value=1, data_type="BOOLEAN",
    score_id=f"fb-{request_id}", # idempotent
)
```

WHAT TO CALL OUT

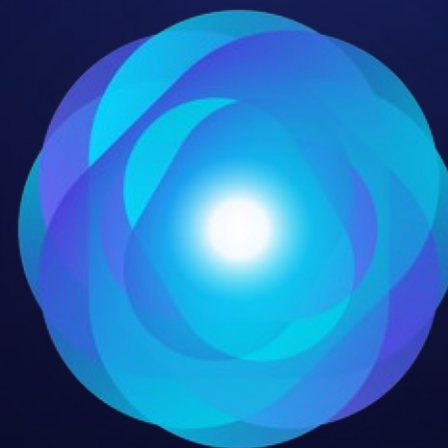
- **A config is a contract**
type, range and categories are validated on write.
- **BOOLEAN for thumbs**
Langfuse's recommended shape for a thumbs vote — 1 or 0.
- **score_id makes re-votes safe**
the second click overwrites; you never double-count.
- **Then filter on it**
Scores ▶ user-feedback < 1 is your unhappy-customer worklist.

07

MODULE 6

Evaluation deep dive

A golden set, rules, judges, DeepEval, human ground truth — then ship v2 and prove it.



Three Levels of Agent Evaluation



OUTCOME

the customer's view

Was the task solved?

completeness

business_rules_ok

goal_accuracy



TRAJECTORY

the engineer's view

Was the path sane?

route_correct

tool_correctness

trajectory_quality



COMPONENT

the debugger's view

Was each step right?

contextual_relevancy

faithfulness

argument_correctness



An agent can be right for the wrong reasons

a correct answer whose supervisor never called the policy specialist is a trajectory failure waiting to become an outcome failure

The Golden Dataset



WHAT 22 GOLDEN ITEMS COVER

6

account

balances, EMI, schedules



5

policy

charges, rules, documents



4

service

tickets and complaints



3

multi-intent

two asks in one message



4

adversarial

rule-breaking, injection



each item stores input · expected_output · metadata

FOUR WAYS TO CREATE ITEMS



SDK

seeded and reproducible — fixed ids upsert



UI

for the expert who won't open a notebook



From production traces

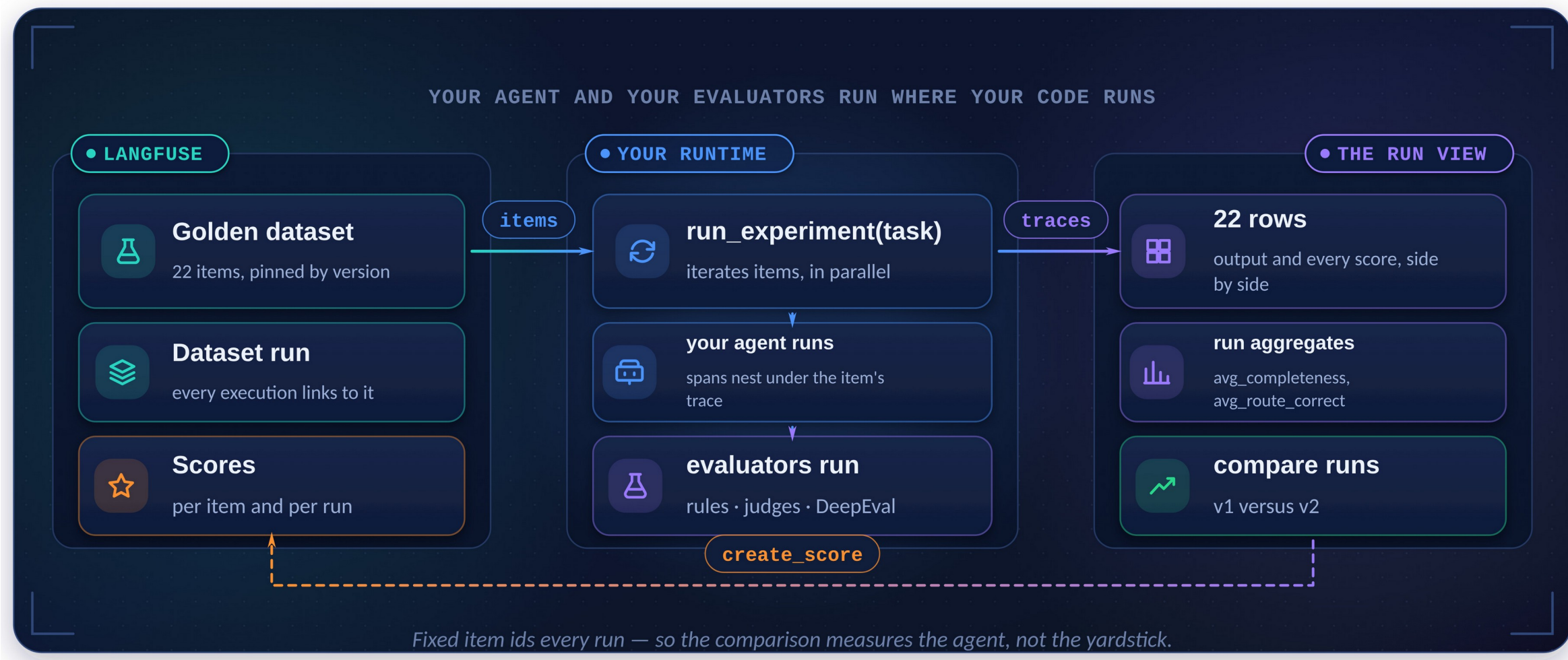
Module 7 promotes real failures



From annotation queues

a reviewed trace's 'Add to dataset'

How an Experiment Run Actually Works



Rule-Based Evaluators · Free, Deterministic, Total



FIVE RULES · ZERO TOKENS · 100% COVERAGE



route_correct

expected routes $\subseteq$ routes visited

TRAJECTORY



tool_correctness

expected tools $\subseteq$ tools called

TRAJECTORY



ticket_schema_ok

the ticket validates against the model

COMPONENT



business_rules_ok

no waiver invented, no rate promised

OUTCOME



completeness

every must-mention group present

OUTCOME

\$0.00

what this whole
pass costs

22 / 22

items scored, no
sampling



They will never drift

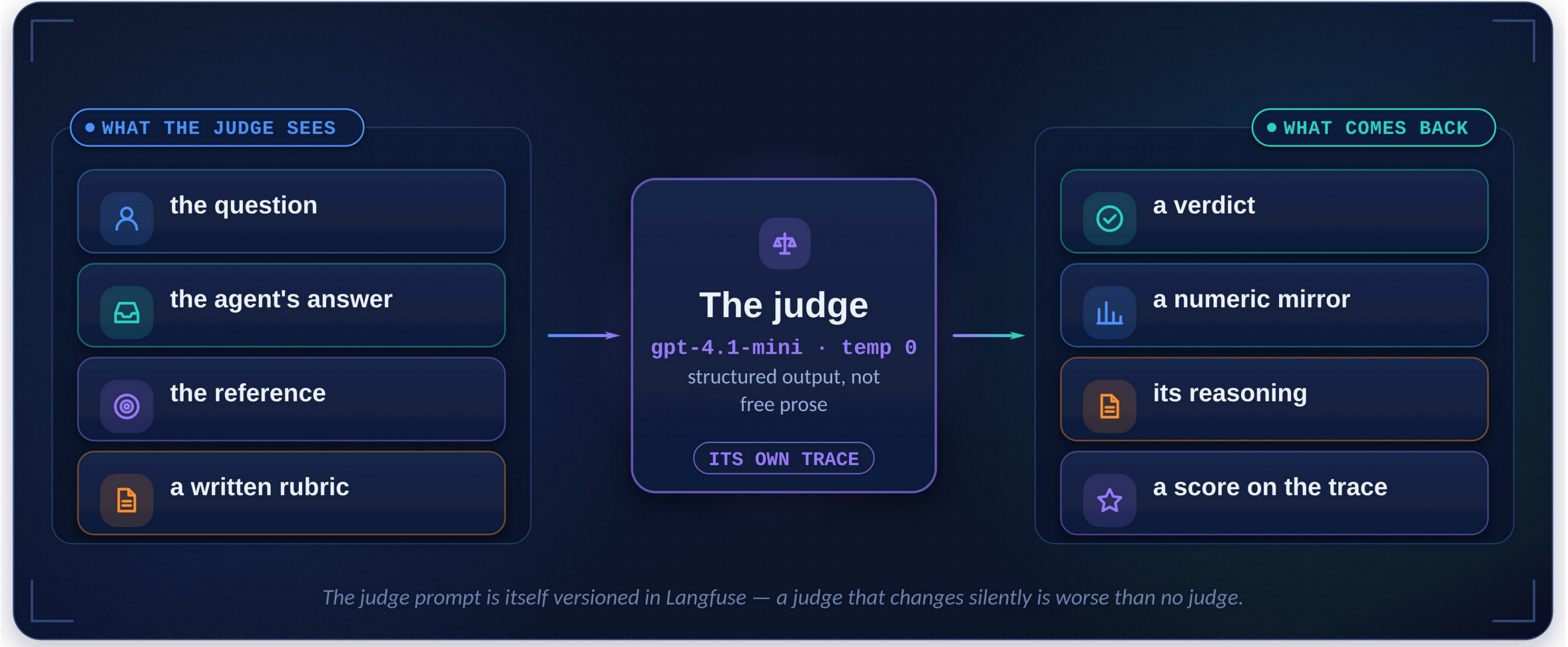
a regex does not have a bad day or quietly
get more generous after a model update



One interface, two places

applied post-hoc to a collected run here,
passed into run_experiment(evaluators=...)
later

How an LLM Judge Works



Four Judge Biases, Four Countermeasures



SELF-PREFERENCE

judges favour their own model family's output

THE COUNTERMEASURE, IN THE CODE YOU RUN
judge 4.1-mini ≠ agent 4o-mini

VERBOSITY

longer answers score higher regardless of content

THE COUNTERMEASURE, IN THE CODE YOU RUN
rubric says ignore length — and we measure `corr(length, score)`

POSITION

in pairwise comparisons the first candidate wins

THE COUNTERMEASURE, IN THE CODE YOU RUN
judge both orders and report the flip rate

DRIFT

the same judge scores differently in three months

THE COUNTERMEASURE, IN THE CODE YOU RUN
pinned model • temperature 0 • judge prompt versioned

A negative length–score correlation is common on v1 — the agent pads with pleasantries exactly when it has nothing to say.

DeepEval · It Computes, Langfuse Stores



DeepEval



a metric library — it computes, and that is all



LLMTestCase



your LangGraph run reshaped into input, output, expected, retrieval context and tools called



create_score

Langfuse stores



pushed exactly like a hand-rolled score

THE MENU, ON THE TWO AXES THAT MATTER

METRIC	LEVEL	REFERENCE
AnswerRelevancy	end-to-end	free
GEval (correctness)	end-to-end	based
TaskCompletion	end-to-end	free
ToolCorrectness	trajectory	based
ArgumentCorrectness	component	free
Faithfulness	component	free
ContextualRelevancy	component	free
Hallucination	end-to-end	based

Careful



Hallucination is the one metric where LOWER is better

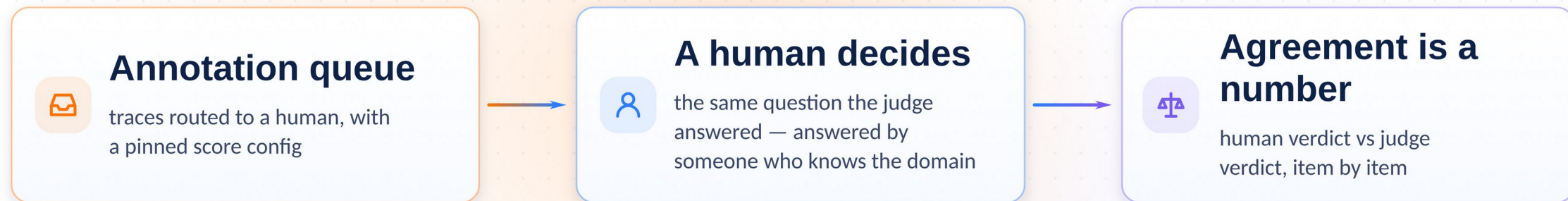
Multi-turn



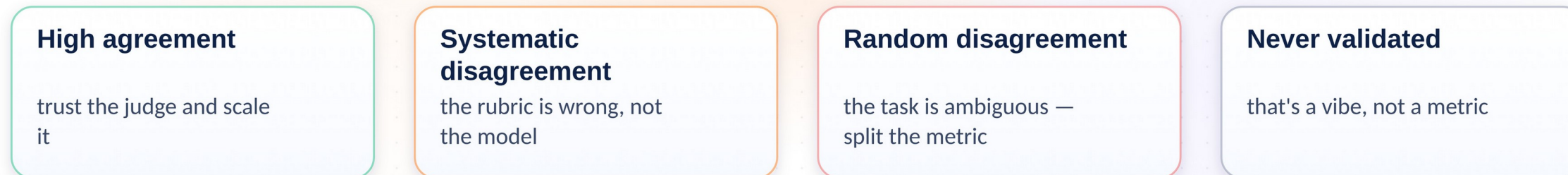
a whole session scored, not one reply

One evaluator interface — rules, judges and DeepEval all return the same Evaluation object.

Human Annotation · Ground Truth for the Judge



WHAT THE AGREEMENT RATE TELLS YOU



v1 → v2 · Fixing What the Scores Diagnosed



Supervisor

route_correct 0.20 → 1.00

v1

answer directly when you can



v2

NEVER answer policy from memory — always route



Policy prompt

completeness 0.44 → 0.86

v1

paraphrases the charges



v2

quote the exact figure and cite the policy id



Service prompt

business_rules 0.50 → 0.95

v1

forgets the ticket id



v2

always echo the ticket id and the document list



Guardrails

adversarial 0.25 → 1.00

v1

no instruction at all



v2

never waive a charge, never promise a rate

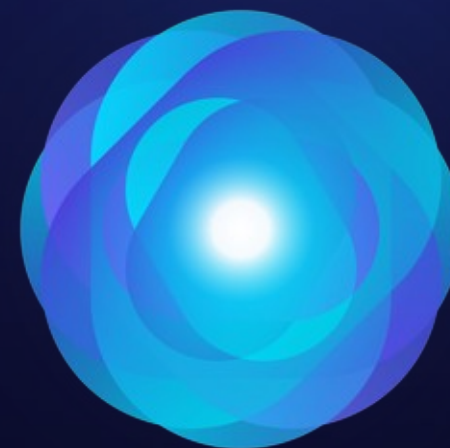
Every edit is a new prompt version, promoted with a label move — never a redeploy.

08

MODULE 7

Live & automatic evaluation

Production has no expected answer. Here is what you score instead — and how it feeds back.



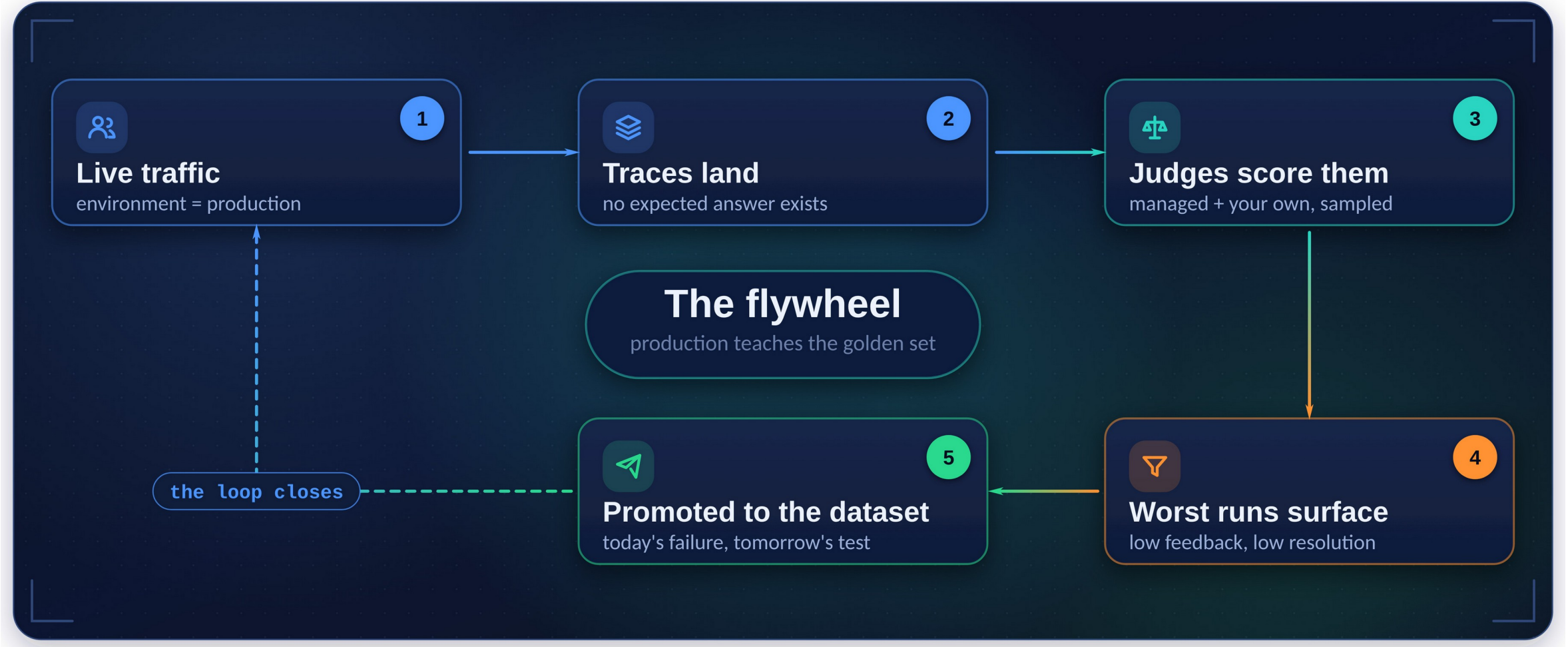
Production Has No expected_output



Everything offline evaluation leans on is gone. These four remain.

- **User feedback**
the only signal that comes from the person who actually needed the answer (Module 5).
- **Reference-free judges**
"does this reply resolve the question?" — anchored rubrics, no ground truth required.
- **Grounding checks**
is the answer supported by what the retriever actually fetched? Faithfulness, cheaply.
- **Business rules**
the same deterministic guardrails from Module 6 — they work identically on live traffic.
- **All of it lands on environment=production**
cleanly separated from today's development traffic, in the same project.

The Production Flywheel



Managed Judge on Live Traffic



Two minutes of UI, then it runs forever without you.

- **Evaluators ► Set up evaluator ► Helpfulness**
a reference-free template from the managed catalog.
- **Target: Live data ► Observations**
filter observation name = meridian-support-agent — the root span carries the whole request.
- **Add environment = production**
otherwise you will score your own dev traffic and wonder why the numbers look odd.
- **Map input → question, output → answer**
the live preview shows real rows, so mapping mistakes are obvious immediately.
- **Sampling 100% today, 5-20% in reality**
from now on every NEW production trace is scored minutes after it arrives.

~2 min

to configure

0

lines of code

minutes

until the first score lands

Your Own Online Scorer



The cron job version — reference-free, and multi-level.

module_7.py

```
class ResolutionVerdict(BaseModel):
    resolved: float = Field(ge=0, le=1)
    reasoning: str

# anchored rubric: 1.0 = specific facts + a next step
#                  0.3 = vague reassurance, no figures

lf.create_score(trace_id=tid, name="resolution_conf",
               value=v.resolved)
lf.create_score(observation_id=oid, name="grounded", value=g)
lf.create_score(session_id=sid, name="conversation_quality", value=q)
```

WHAT TO CALL OUT

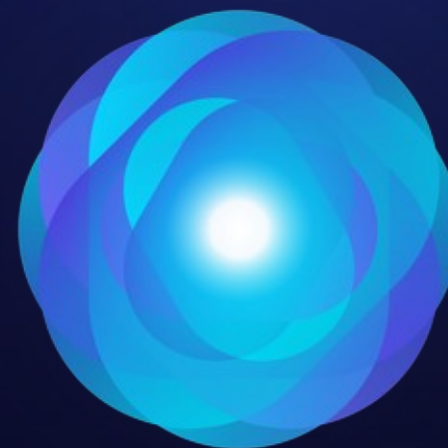
- **Anchor the rubric numerically**
'1.0 means X, 0.3 means Y' — not 'rate 0 to 1'.
- **Score three different levels**
trace, observation and session take different arguments.
- **Backfill is the same call**
point it at a timestamp window and score history.
- **Sessions need multi-turn metrics**
ConversationalGEval judges consistency across turns.

09

MODULE 8

Custom dashboards & metrics

The wall your team actually looks at — and the API behind every number on it.



The Four Widgets Worth Having



WHAT ACTUALLY GOES ON THE WALL

Cost per day

SOURCE Observations
METRIC totalCost · sum by day
FILTER environment = production

User feedback trend

SOURCE Scores (numeric)
METRIC value · avg of user-feedback
FILTER environment = production

p95 latency

SOURCE Observations
METRIC latency · p95 by day
FILTER type = GENERATION

Escalation rate

SOURCE your own definition
METRIC conversations that raised a ticket
FILTER a business KPI, not a platform metric

The Metrics API · One Query Object



ONE QUERY OBJECT, EVERY NUMBER ON YOUR DASHBOARD

view

observations · traces · scores
what kind of row am I counting?

metrics

measure + aggregation
totalCost·sum · latency·p95 ·
count·count

dimensions

group by
model · name · environment · user

filters

column, operator, value
type = GENERATION · environment =
production

timeDimension

granularity
hourly · daily buckets

from / toTimestamp

the window
ISO-8601, always explicit



Budget note

the Metrics API allows 100 requests per day on the free tier — this workshop uses about a dozen

3 calls

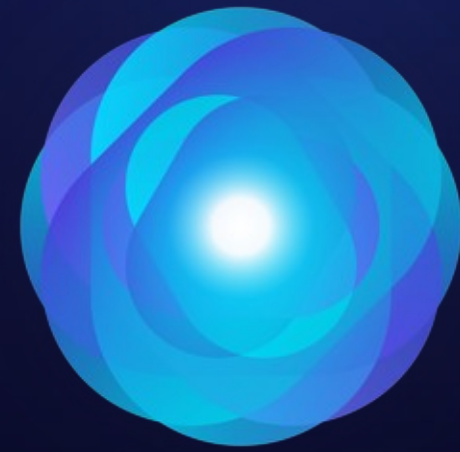
is all a health check needs

10

MODULE 9

Monitoring & alerting

A baseline, a scheduled check, a real incident — and the rollback that closes it.



Scores · Who Writes Them, What They Hang Off



WHO WRITES A SCORE



Rule-based code

deterministic, free



LLM-as-a-judge

a rubric, applied by a model



Human annotation

a reviewer in a queue



Your application

the thumbs the customer clicked

SCORE

name · value · comment

NUMERIC

0.0 – 1.0, or any range

CATEGORICAL

achieved / partial / failed

BOOLEAN

the thumbs-up, as 1 or 0

TEXT

notes only — no analytics

attaches to

WHAT IT HANGS OFF



a trace

the whole request



an observation

one span — e.g. the retriever



a session

the whole conversation



a dataset run

the experiment's report card

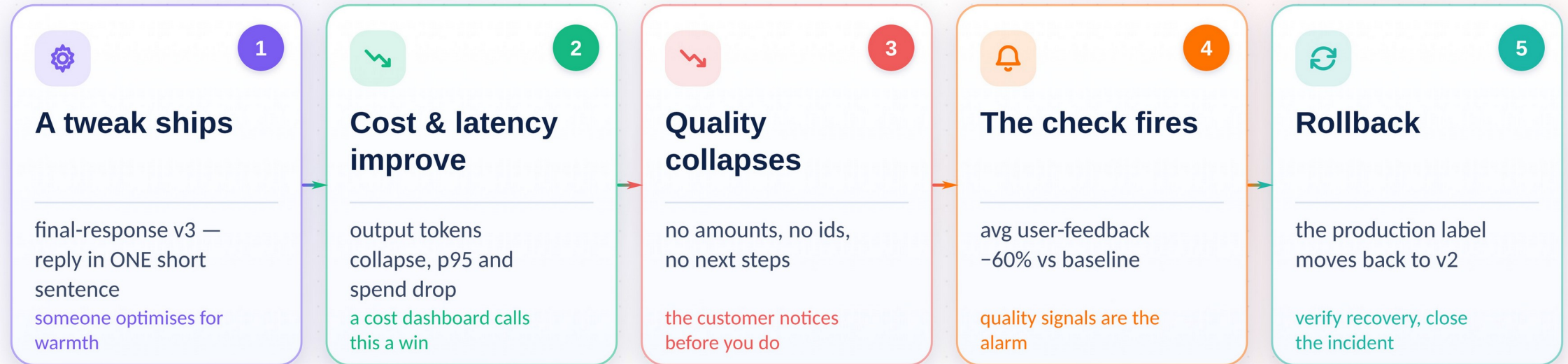
Baseline → Check → Alert



The Incident



THE INCIDENT, IN FIVE MOVES



A cost-only dashboard would have called this deploy an improvement



the deploy got cheaper and faster while getting materially worse. Latency and spend are necessary signals — they are not sufficient ones. Only a quality metric catches this class of regression.

Why a Cost Dashboard Would Have Shipped It



SAME DEPLOY, THREE DASHBOARDS



p95 latency

-34%

3 180 ms → 2 100 ms

LOOKS LIKE A WIN



cost per request

-41%

\$0.00043 → \$0.00025

LOOKS LIKE A WIN



avg user-feedback

-62%

0.81 → 0.31

THE TRUTH



avg resolution score

-48%

0.79 → 0.41

THE TRUTH



Two dashboards said ship it. Two said roll it back.

this is exactly why the health check thresholds are written on quality signals first — feedback and resolution — with latency and cost as supporting evidence, never as the verdict.

Monitors, Automations, and Closing the Incident



The notebook job is one way. Langfuse runs the same idea server-side.

- **Monitors ► + New monitor**
data source Scores (numeric) → avg of user-feedback → filter environment = production.
- **A lookback window and a threshold**
e.g. 1 hour, alert below 0.5 — with an optional warning threshold before the alert.
- **Notification through Automations**
Slack, a signed webhook, or a GitHub Action. Webhook payloads carry an HMAC signature.
- **They fire on state transitions**
OK → WARNING → ALERT and back. One page per incident, not one page per minute.
- **Rollback, then verify recovery**
move the production label to v2, generate traffic, re-score, and watch the metric return.

< 0.5

the feedback threshold

1 hour

the lookback window

1 call

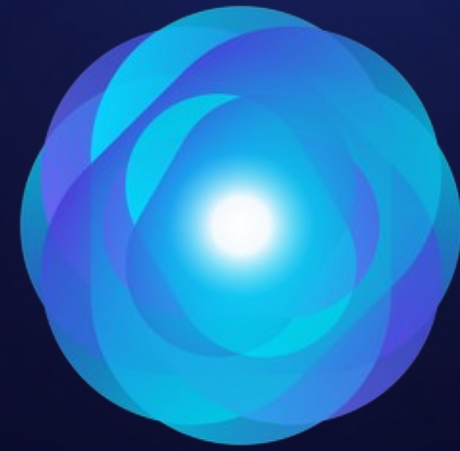
to roll back

11

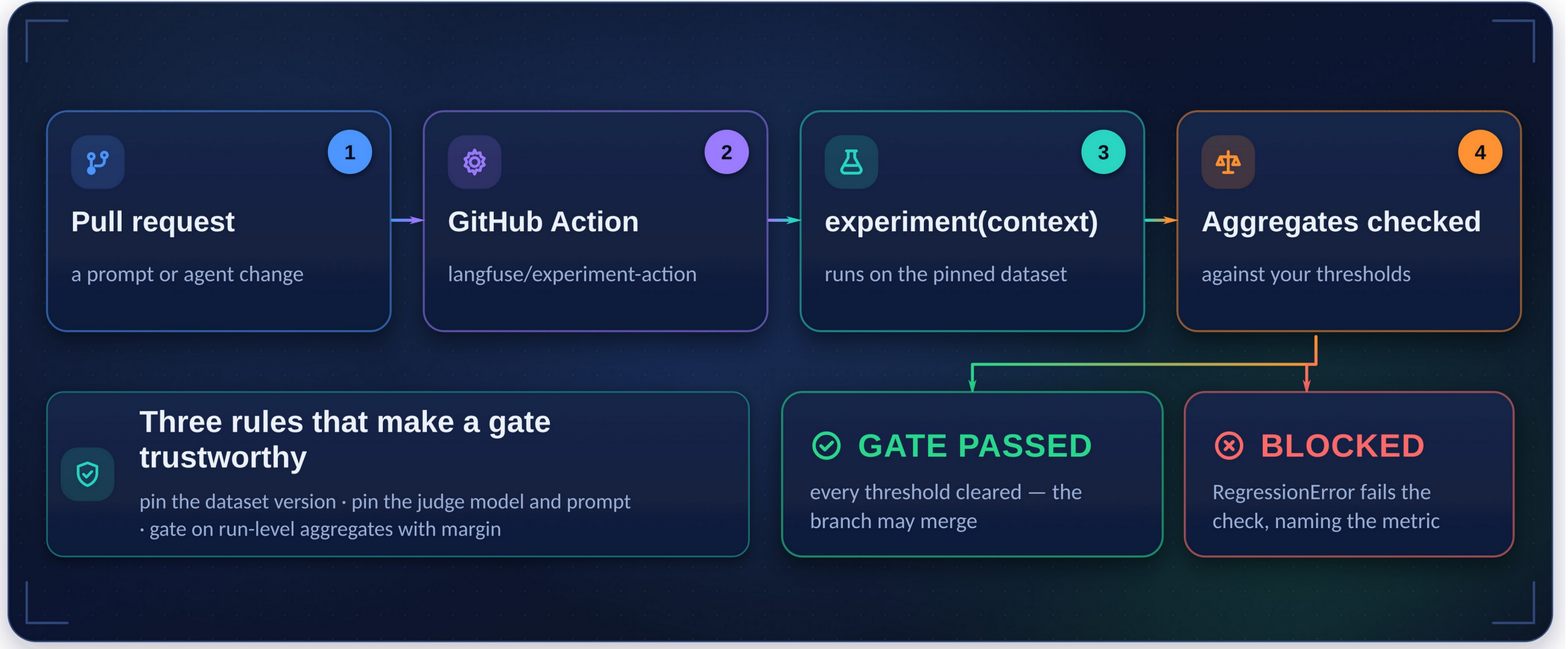
MODULE 10

Quality gates in CI/CD

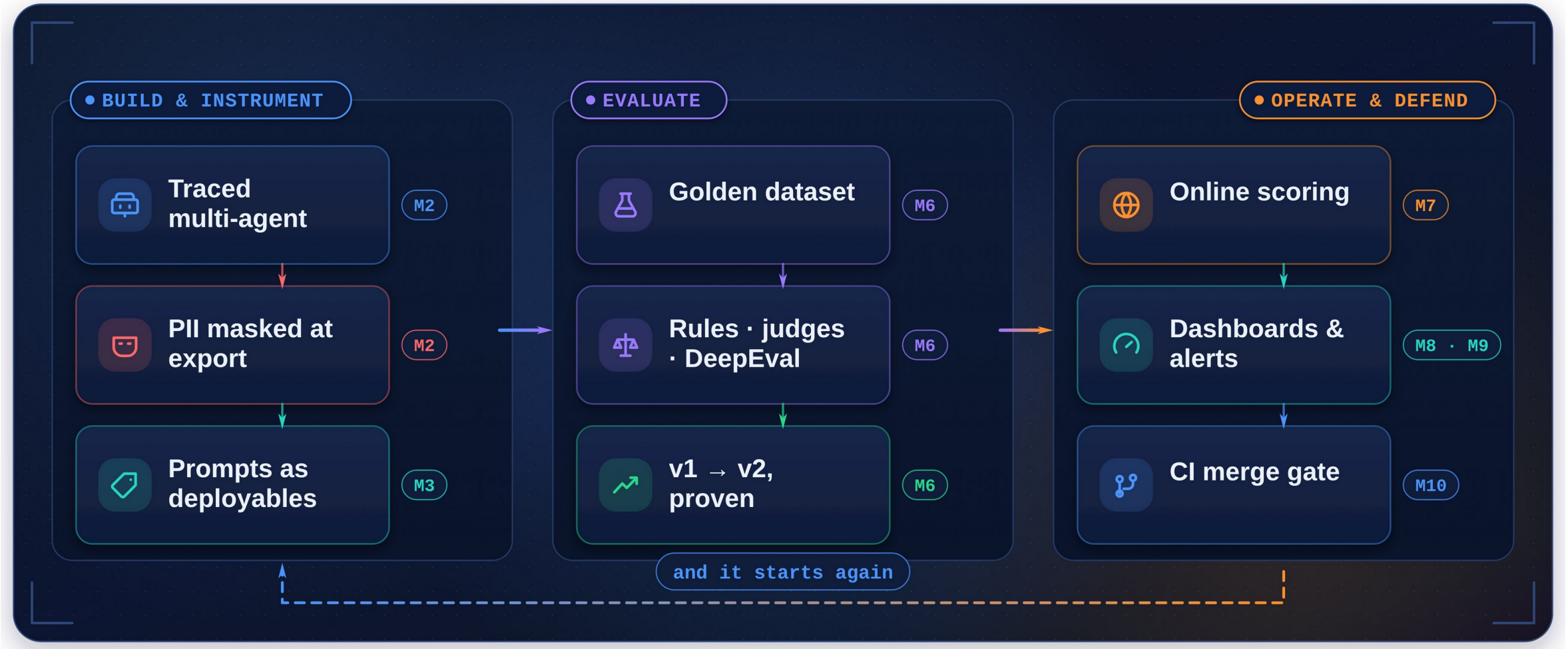
The Module 6 experiment becomes a merge gate — and the regression never reaches main.



The Merge Gate



The Complete Production Loop



Thank You

